

Riassunto Dettagliato e Aggiornato:
*An improved frontier-based robot exploration strategy combined
with deep reinforcement learning*
Robotics and Autonomous Systems 181 (2024) 104783

NECKER

Basato sul PDF originale + qualche precisazione

9 Dicembre 2025

Sommario

Questo documento presenta un'analisi dettagliata di una nuova strategia di esplorazione autonoma per robot mobili. L'approccio proposto combina il metodo tradizionale basato sulle frontiere con il Deep Reinforcement Learning (DRL) per la selezione ottimale dei punti target. Il metodo introduce una rete neurale innovativa (LFPN) che elabora sia una mappa globale ridimensionata che mappe locali a risoluzione originale, mitigando la perdita di informazioni causata dal campionamento delle mappe. I risultati sperimentali, sia in simulazione che su robot reali, dimostrano che questo metodo riduce significativamente la distanza di viaggio necessaria per completare l'esplorazione rispetto agli approcci esistenti.

Indice

1	Introduzione	2
2	Panoramica sulle Metodologie	2
3	Metodologia Proposta	3
3.1	Architettura del Sistema	3
3.2	Rilevamento delle Frontiere e Processo Decisionale	3
3.2.1	Identificazione e Filtraggio dei Candidati	3
3.2.2	Criterio di Selezione: Massimizzazione del Valore Q	4
3.3	Struttura della Rete Neurale (LFPN)	5
3.3.1	1. Rete Globale (Global Network)	5
3.3.2	2. Rete Locale (Local Network)	6
3.3.3	3. Rete di Output (Valutazione)	7
3.4	Processo Decisionale Dettagliato	7
4	Ambiente di Reinforcement Learning	8
4.1	Addestramento della Rete tramite Double Deep Q-Learning (DDQN)	8
4.1.1	Configurazione del Dual Network	8
4.1.2	Funzione di Perdita e Ottimizzazione	8
4.1.3	Aggiornamento Soft Target	8
4.1.4	Gestione dei Dati e Training Loop	9
5	Esperimenti e Risultati	10
5.1	Setup Sperimentale	10
5.2	Risultati Comparativi	10
5.3	Esperimenti Fisici	10

1 Introduzione

L'esplorazione autonoma richiede che un robot si muova in un ambiente sconosciuto costruendo una mappa utilizzabile per la navigazione successiva. L'obiettivo principale è minimizzare la distanza percorsa pur costruendo una mappa completa. Tradizionalmente, i metodi *frontier-based* guidano il robot verso i confini tra spazio libero e spazio sconosciuto (frontiere). Tuttavia, metodi semplici come la scelta della frontiera più vicina tramite Depth-First-Search (DFS) possono portare a percorsi inefficienti o collisioni se gli ostacoli sono vicini al target.

Con l'avvento del Deep Learning (DL) e del Deep Reinforcement Learning (DRL), sono state proposte soluzioni per gestire l'alta dimensionalità dei problemi decisionali robotici. Tuttavia, i metodi DRL esistenti spesso richiedono il ridimensionamento della mappa per adattarsi all'input della rete neurale, causando una perdita di risoluzione e informazioni critiche, specialmente in ambienti vasti.

Il metodo proposto in questo studio introduce una strategia migliorata che utilizza una rete neurale per estrarre caratteristiche sia globali che locali, permettendo al robot di "comprendere" meglio l'ambiente e completare l'esplorazione con un costo di distanza inferiore.

2 Panoramica sulle Metodologie

Prima del deep learning, il problema era trattato come un processo decisionale di Markov parzialmente osservabile (POMDP).

- **Metodi Tradizionali:** Yamauchi (1997) ha introdotto l'esplorazione basata su frontiere. Gonzalez-Baños (2002) ha migliorato la sicurezza selezionando target nello spazio libero sicuro. Altri metodi, come quello di Bourgault et al., si basano sulla teoria dell'informazione e sulla massimizzazione del guadagno informativo.
- **Metodi basati su DRL:** Niroui et al. hanno proposto l'uso di DRL con reti ricorrenti (RNN), ma il loro metodo richiede il ridimensionamento della mappa globale, perdendo dettagli. Chen et al. hanno utilizzato mappe locali fisse attorno al robot, perdendo però il contesto globale dell'ambiente.

3 Metodologia Proposta

Il nucleo della strategia proposta è l'integrazione di un sistema SLAM tradizionale con un decisore basato su Deep Reinforcement Learning (DRL). Per superare i limiti delle architetture esistenti, che spesso soffrono di perdita di informazioni durante il ridimensionamento delle mappe o mancano di consapevolezza globale, viene introdotta la **LFPN** (Local Feature Pyramid Network). Questa architettura ibrida elabora simultaneamente una visione globale compressa e dettagli locali ad alta risoluzione, permettendo al robot di valutare i target di esplorazione con una comprensione contestuale completa.

3.1 Architettura del Sistema

L'architettura complessiva è modulare e comprende tre componenti interconnessi:

1. **Modulo SLAM (Mappatura e Localizzazione):** Viene impiegato l'algoritmo *Cartographer* per elaborare i dati dei sensori (Lidar) in tempo reale. Il risultato è una mappa a griglia di occupazione 2D (Occupancy Grid), dove ogni cella possiede uno stato discreto: spazio libero, spazio occupato (ostacolo) o spazio sconosciuto. Questo modulo fornisce costantemente la posa del robot e la mappa aggiornata m_c .
2. **Modulo di Navigazione:** Una volta selezionato un target dalla strategia DRL ([vista nelle sezioni successive](#)), la navigazione avviene su due livelli:
 - **Pianificazione Globale:** L'algoritmo A^* calcola il percorso ottimale privo di collisioni dalla posizione attuale al target sulla mappa statica.
 - **Pianificazione Locale:** Il pianificatore *Timed Elastic Band* (TEB) ottimizza la traiettoria locale in tempo reale, considerando la cinematica del robot e gli ostacoli dinamici, inviando i comandi di velocità al controller dei motori.
3. **Strategia di Esplorazione (DRL):** Questo modulo agisce come il cervello decisionale. Ad ogni step, analizza la mappa parziale costruita, individua i punti di frontiera candidati e utilizza la rete LFPN per assegnare un punteggio di utilità a ciascuno, determinando la prossima destinazione.

3.2 Rilevamento delle Frontiere e Processo Decisionale

Il processo che porta il robot a muoversi non si basa su una semplice scansione geometrica, ma su una pipeline strutturata che trasforma dati grezzi in decisioni strategiche volte a minimizzare il costo di esplorazione.

3.2.1 Identificazione e Filtraggio dei Candidati

Il primo passo consiste nell'individuare dove il robot *potrebbe* andare. Questo processo avviene sulla griglia di occupazione 2D costruita dal modulo SLAM.

- **Definizione di Frontiera:** Una frontiera è definita come il bordo che separa lo spazio libero (free space) dallo spazio sconosciuto (unknown space) nella mappa.
- **Analisi delle Componenti:** Il sistema analizza la mappa per individuare i bordi tra spazio libero e sconosciuto. Successivamente, applica l'analisi delle componenti connesse per determinare il centro di ciascun bordo, che diventa un potenziale punto target.
- **Filtraggio del Rumore (Noise Reduction):** Per evitare che il robot tenti di esplorare falsi spazi generati dal rumore dei sensori, vengono scartate tutte le frontiere che presentano una lunghezza del bordo inferiore a 0.5 m.
- **Spazio Sicuro (Safe Space):** Il robot non viene inviato esattamente sul bordo della frontiera, poiché potrebbe essere troppo vicino a ostacoli. Lo spazio libero viene eroso per ottenere un sottoinsieme definito "Spazio Sicuro". Il punto candidato finale (azione) viene scelto selezionando la posizione nello spazio sicuro più vicina al centro del bordo della frontiera.

3.2.2 Criterio di Selezione: Massimizzazione del Valore Q

Una volta ottenuto l'insieme dei candidati $c = [p_1, \dots, p_N]$, il sistema deve scegliere il migliore. A differenza dei metodi tradizionali che spesso scelgono il punto più vicino (Nearest Frontier), il metodo LFPN sceglie in base all'efficienza complessiva prevista.

La Funzione Obiettivo

L'obiettivo matematico dell'esplorazione è minimizzare la distanza totale percorsa per completare la mappa. Il problema è modellato come un processo decisionale di Markov (MDP) dove la ricompensa r ricevuta ad ogni passo è pari all'opposto della distanza percorsa l :

$$r_i = -l_i \quad (1)$$

Di conseguenza, massimizzare la ricompensa cumulativa equivale a minimizzare la distanza totale di viaggio.

Interpretazione del Valore Q e Stabilità dell'Apprendimento

Per comprendere appieno il funzionamento del sistema, è necessario chiarire la distinzione tra la ricompensa fisica immediata e la stima strategica della rete, nonché il meccanismo che garantisce la stabilità dell'apprendimento.

Reward Immediato vs. Valore Q: La Lungimiranza della Rete

Spesso si crea confusione tra il *reward* ricevuto e l'output della rete.

- **Reward Immediato (r):** È un valore fisico reale. Se il robot percorre 5 m per raggiungere una frontiera, il reward è $r = -5$. Questo valore misura solo il costo dell'azione corrente.
- **Valore Q (Output):** È una previsione a lungo termine. Il valore Q rappresenta la somma del reward immediato più tutti i reward futuri stimati fino al completamento della mappa:

$$Q \approx r_{\text{ora}} + (r_{\text{futuro}_1} + r_{\text{futuro}_2} + \dots)$$

Esempio Pratico: Supponiamo che il robot abbia due opzioni:

1. **Frontiera A (Vicina, 5 m):** È un vicolo cieco. Andarci costa poco subito (-5), ma poi costringerà il robot a tornare indietro e fare un giro lungo (costo futuro -95). Il valore Q totale sarà -100 .
2. **Frontiera B (Lontana, 20 m):** È l'ingresso di un corridoio centrale. Andarci costa molto subito (-20), ma posiziona il robot strategicamente per finire il lavoro velocemente (costo futuro -60). Il valore Q totale sarà -80 .

Il sistema LFPN sceglierà la **Frontiera B** perché $-80 > -100$ (essendo numeri negativi, -80 è migliore), dimostrando che la rete accetta un costo immediato maggiore per ottenere un risparmio totale nel lungo periodo.

La rete apprende questa stima combinando la visione d'insieme della **Rete Globale** (che aiuta a pianificare l'ordine delle stanze/aree) con i dettagli della **Rete Locale** (che valuta l'accessibilità immediata). (affrontate nella prossima sezione)

Selezione Greedy

Al termine della valutazione, il sistema adotta una politica *greedy* (avida). Viene selezionato come prossimo target p_g il candidato p_t che possiede il valore Q massimo tra tutte le opzioni disponibili:

$$p_g = p_t \quad \text{dove} \quad t = \arg \max(q_t) \quad (2)$$

Questo punto viene quindi inviato al pianificatore globale A^* per la navigazione.

3.3 Struttura della Rete Neurale (LFPN)

La sfida principale nell'uso di reti neurali per l'esplorazione è la variabilità delle dimensioni della mappa. Ridimensionare l'intera mappa a una dimensione fissa riduce drasticamente la risoluzione, cancellando dettagli cruciali per la sicurezza (piccoli ostacoli) e per l'esplorazione (passaggi stretti). La LFPN risolve questo problema strutturandosi in tre sottoreti distinte:

3.3.1 1. Rete Globale (Global Network)

Questa rete ha lo scopo di estrarre il contesto generale dell'ambiente e la topologia delle aree inesplorate.

- **Pre-processing dell'Input:** La mappa parziale attuale non viene inserita grezza. Il sistema ritaglia la "regione efficace" (il rettangolo che contiene lo spazio già esplorato), la scala per farlo fittare nella dimensione richiesta (300×300), quindi se è più piccola la ingrandisce e se è più grande la rimpicciolisce. Infine applica un padding con valori di "spazio sconosciuto" se nel rettangolo esistono aree ancora inesplorate.
- **Rappresentazione del Tensore:** L'input è un tensore a 5 canali ($5 \times 300 \times 300$), dove ogni canale è una mappa binaria (valori 0 o 1) che rappresenta:
 1. Spazio sconosciuto.
 2. Spazio libero.
 3. Spazio occupato (ostacoli).
 4. Posizioni di tutte le frontiere rilevate.
 5. Posizione attuale del robot.
- **Architettura Interna:** La rete utilizza strati convoluzionali seguiti da attivazioni ReLU. Per mitigare il problema del "campo percettivo limitato" tipico delle CNN standard (che faticano a collegare informazioni spazialmente distanti), viene inserito un **Self-Attention Module** (ispirato ai blocchi non-local). Questo modulo calcola le relazioni tra pixel distanti, permettendo alla rete di comprendere la struttura globale.
- **Output:** Un vettore di caratteristiche globali di dimensione 1×200 .

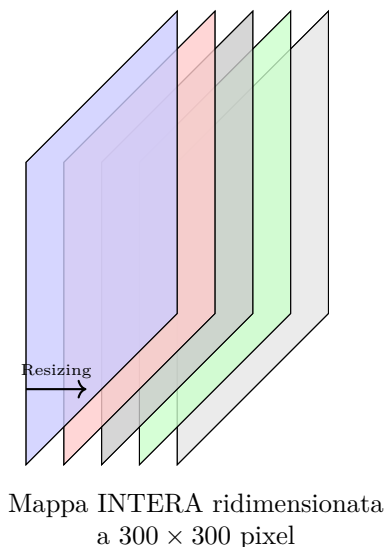
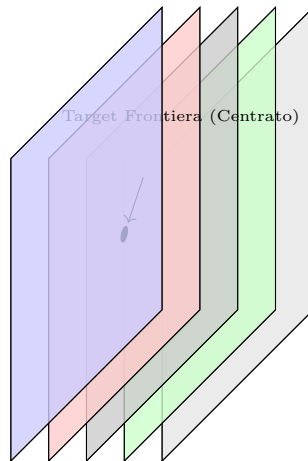


Figura 1: Rappresentazione del Tensore di Input Globale

3.3.2 2. Rete Locale (Local Network)

Questa rete analizza i dettagli fini attorno a un candidato specifico, garantendo che il robot consideri la geometria locale (es. vicinanza a ostacoli o vicoli ciechi).

- **Input a Risoluzione Originale:** Per ogni frontiera candidata, viene estratta una porzione di mappa (crop) di 300×300 pixel centrata esattamente sulla frontiera stessa. Fondamentalmente, **non viene applicato alcun ridimensionamento (scaling)**, preservando la risoluzione originale della griglia di occupazione.
- **Gestione dei Bordi:** Se il crop esce dai confini della mappa conosciuta, l'area esterna viene riempita (padded) con il valore di "spazio sconosciuto".
- **Struttura del Tensore:** Identica alla rete globale (5 canali). Tuttavia, nel canale della "Posizione del Robot", sarà presente nella griglia un 1 (pos del robot) solo se il robot si trova fisicamente all'interno del crop locale; altrimenti è una matrice tutta a 0. Nel canale delle frontiere, il centro del crop è sempre segnato come 1 (essendo la frontiera sotto esame).
- **Condivisione dei Pesì:** Sebbene la Rete Locale abbia la stessa identica architettura strutturale della Rete Globale (inclusi i blocchi di attenzione), essa possiede parametri (pesi) distinti, addestrati specificamente per riconoscere feature locali.
- **Output:** Un vettore di caratteristiche locali di dimensione 1×200 per la specifica frontiera analizzata.



Ritaglio (Crop) 300×300
Risoluzione ORIGINALE (No Scaling)

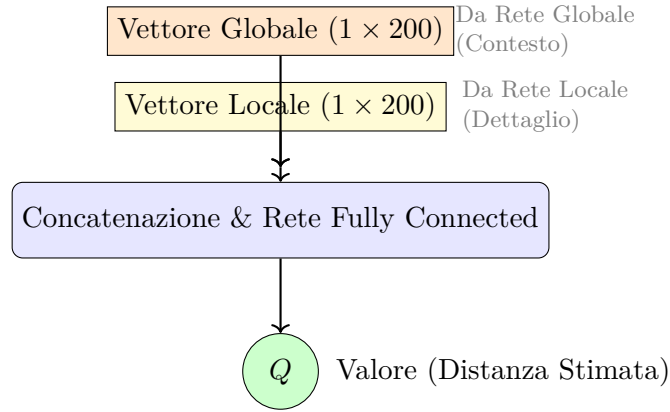
3.3.3 3. Rete di Output (Valutazione)

Questa rete fonde le informazioni globali e locali per produrre il valore Q (valore atteso della ricompensa).

- **Input:** Riceve in ingresso la concatenazione del vettore globale (1×200) e del vettore locale (1×200), risultando in un vettore di input 1×400 .
- **Architettura Fully Connected:** Il vettore passa attraverso una serie di strati lineari (densamente connessi) che riducono progressivamente la dimensionalità:

$$\text{Linear}(400 \rightarrow 300) \xrightarrow{\text{ReLU}} \text{Linear}(300 \rightarrow 200) \xrightarrow{\text{ReLU}} \text{Linear}(200 \rightarrow 100) \xrightarrow{\text{ReLU}} \text{Linear}(100 \rightarrow 1)$$

- **Output Finale:** Un singolo scalare che rappresenta il valore di valutazione Q per quella specifica frontiera (quanto è lontano il robot da quella frontiera es: lontano 10 metri $\rightarrow$ **Reward = -10**).



3.4 Processo Decisionale Dettagliato

Il ciclo decisionale viene eseguito ogni volta che il robot raggiunge un target o necessita di una nuova destinazione. La procedura è rigorosa:

1. **Rilevamento Frontiere:** Il sistema scansiona la mappa attuale m_c per identificare i bordi tra spazio libero e sconosciuto. Vengono applicati filtri per rimuovere frontiere troppo piccole (rumore sensoriale) o irraggiungibili. Si ottiene un insieme di candidati $c = [p_1, p_2, \dots, p_N]$.
2. **Elaborazione Globale:** Viene generata la mappa ridimensionata m_r e passata attraverso la Rete Globale G per ottenere il vettore di contesto $v_g = G(m_r)$. Questo calcolo viene effettuato *una sola volta* per passo decisionale, poiché il contesto globale è identico per tutti i candidati.
3. **Valutazione Iterativa dei Candidati:** Per ogni frontiera p_i nell'insieme c :
 - (a) Si ritaglia la mappa locale m_{Li} centrata su p_i .
 - (b) Si calcola il vettore delle caratteristiche locali $v_{Li} = L(m_{Li})$ tramite la Rete Locale.
 - (c) Si calcola il punteggio $q_i = O(v_g, v_{Li})$ tramite la Rete di Output, fondendo il contesto globale statico con il dettaglio locale specifico.
4. **Selezione Greedy:** Dopo aver valutato tutti gli N candidati, il sistema seleziona il punto p_g associato al valore q massimo: $p_g = p_k$ dove $k = \text{argmax}(q)$ (es: $-10 > -39$), come visto nella sezione precedente.

4 Ambiente di Reinforcement Learning

4.1 Addestramento della Rete tramite Double Deep Q-Learning (DDQN)

Mentre la sezione precedente ha descritto l'architettura della LFPN come il "cervello" del robot, questa sezione spiega come tale cervello viene istruito. La rete LFPN funge da approssimatore della funzione valore $Q(s, a; \theta)$, dove θ rappresenta i pesi della rete neurale. L'obiettivo dell'addestramento è ottimizzare questi pesi affinché il valore Q in uscita rispecchi fedelmente la distanza negativa futura, permettendo all'algoritmo *greedy* descritto in precedenza di selezionare effettivamente il percorso migliore.

Per garantire la stabilità dell'apprendimento in un ambiente complesso come l'esplorazione autonoma, viene adottato l'algoritmo **Double Deep Q-Learning (DDQN)**.

4.1.1 Configurazione del Dual Network

Il metodo DDQN affronta il problema della sovrastima dei valori Q tipico del Q-Learning standard disaccoppiando la selezione dell'azione dalla sua valutazione. A tale scopo, vengono mantenute due istanze della rete LFPN, identiche nella struttura ma con ruoli e parametri distinti:

- **Policy Network (θ):** È la rete principale utilizzata per prendere decisioni durante l'esplorazione e viene aggiornata costantemente tramite discesa del gradiente. Evolve molto in fretta e quindi sbaglia spesso (ogni passo guarda la previsione della network e calcola di quanto ha sbagliato adattandosi), viene usata per muoversi durante l'addestramento, durante l'esecuzione viene congelata e usata per muoversi.
- **Target Network (θ'):** È una copia più stabile della rete, utilizzata per calcolare il valore obiettivo durante il calcolo della perdita (loss), prevenendo oscillazioni nell'apprendimento. Si evolve molto lentamente aggiornandosi una volta a episodio apprendendo il 3% della **Policy Network** e correggendola ad ogni step, in fase di esecuzione viene scartata.

Inizialmente, i parametri θ e θ' vengono inizializzati casualmente con gli stessi valori.

4.1.2 Funzione di Perdita e Ottimizzazione

Durante ogni passo di addestramento, l'agente preleva un batch di esperienze dal buffer (descritto in seguito) e cerca di minimizzare l'errore tra la previsione della rete corrente e il target calcolato. La peculiarità del DDQN risiede nella formula del target: la *Policy Network* decide quale sia l'azione migliore a'_{max} nello stato successivo s' , mentre la *Target Network* ne calcola il valore.

L'azione ottimale successiva viene selezionata come:

$$a'_{max} = \underset{a}{\operatorname{argmax}} Q(s', a; \theta) \quad (3)$$

La funzione di perdita (Loss Function) $J(\theta)$ minimizzata tramite discesa del gradiente è definita come:

$$J(\theta) = (R(s, a) + \gamma Q(s', a'_{max}; \theta') - Q(s, a; \theta))^2 \quad (4)$$

Se l'episodio termina nello stato successivo, il termine futuro viene omissso e il target è semplicemente la ricompensa immediata $R(s, a)$.

4.1.3 Aggiornamento Soft Target

A differenza degli approcci tradizionali che aggiornano la Target Network copiando bruscamente i pesi ogni N passi, gli autori adottano il metodo *Soft Target Update*, ispirato all'algoritmo DDPG. Questo garantisce un aggiornamento molto più fluido e stabile. Al termine di ogni episodio, i parametri della Target Network vengono aggiornati secondo la formula:

$$\theta' \leftarrow \tau(\theta - \theta') + \theta' \quad (5)$$

Il fattore di aggiornamento soft τ è impostato a 0.03, il che significa che la rete Target cambia molto lentamente, inseguendo la Policy Network con inerzia controllata.

4.1.4 Gestione dei Dati e Training Loop

L'efficienza del training è massimizzata attraverso una gestione aggressiva dei dati:

- **Esperienza (Replay Buffer):** Le interazioni del robot vengono salvate come tuple $\langle s, a, s', r, e \rangle$ in un buffer D^{Replay} , dove e è un booleano che indica la fine dell'episodio.
- **Frequenza di Training:** Per accelerare la convergenza, la rete non viene addestrata solo a fine episodio, ma vengono campionati 50 dataset casuali dal buffer **dopo ogni singolo passo di esplorazione**. Questo approccio intensivo spiega la rapida convergenza osservata sperimentalmente intorno all'episodio 300.
- **Data Augmentation:** Per migliorare la capacità di generalizzazione della rete LFPN (affinché riconosca corridoi e stanze indipendentemente dall'orientamento), le mappe estratte dal buffer vengono ruotate di 90° , 180° o capovolte orizzontalmente/verticalmente prima di essere usate per il training.

L'intero processo è stato eseguito su hardware dedicato (NVIDIA RTX 2080 Ti) per 600 episodi totali (circa 80 ore), utilizzando 4 mappe di dimensioni comparabili per mantenere coerente la scala delle ricompense.

5 Esperimenti e Risultati

5.1 Setup Sperimentale

Il metodo è stato validato su 5 mappe di test con dimensioni e rapporti d'aspetto molto vari (da $30 \times 14 \text{ m}^2$ a $28 \times 60 \text{ m}^2$). Sono stati confrontati i seguenti metodi:

1. **LFPN (Proposto)**
2. **DRL1 (Niroui et al.):** Ridimensiona la mappa globale a 64×64 , usa pesi per i gruppi di celle.
3. **DRL2 (Chen et al.):** Usa solo mappe locali di 20 m attorno al robot e 50 target casuali.
4. **Nearest Frontier:** Sceglie la frontiera più vicina.
5. **Hybrid:** Combina costo di viaggio e guadagno informativo con vari pesi.
6. **Nearest Frontier + RR:** Ricontrolla costantemente il target migliore durante il movimento.

5.2 Risultati Comparativi

- **Distanza Percorsa:** Il metodo LFPN ha completato l'esplorazione con una distanza inferiore rispetto a tutti gli altri metodi in tutte le mappe di test.
- **Grandi Ambienti:** Il vantaggio di LFPN è più evidente in ambienti vasti (Mappa 5). Mentre DRL1 e DRL2 esplorano velocemente all'inizio, finiscono per percorrere distanze totali maggiori.
- **Generalizzazione:** Nonostante il training su mappe piccole, LFPN ha mostrato un'eccellente capacità di generalizzazione su mappe grandi grazie alla sua architettura che preserva i dettagli locali.

5.3 Esperimenti Fisici

È stato condotto un esperimento reale al 9° piano della Scuola di Automazione (area $56 \times 55 \text{ m}^2$) usando un robot equipaggiato con ruote Mecanum e RPLIDAR.

- Il robot ha completato l'esplorazione reale percorrendo 445 m, un valore molto simile ai 430 m ottenuti nella simulazione dello stesso ambiente.
- Questo dimostra che il metodo è facilmente trasferibile dall'ambiente simulato a quello reale (Sim-to-Real).

Riferimenti bibliografici

- [1] Rui Wang, Jie Zhang, Ming Lyu, Cheng Yan, Yaowei Chen (2024). *An improved frontier-based robot exploration strategy combined with deep reinforcement learning. Robotics and Autonomous Systems*, 181, 104783.
<https://doi.org/10.1016/j.robot.2024.104783>